

Week 6 Problem Set

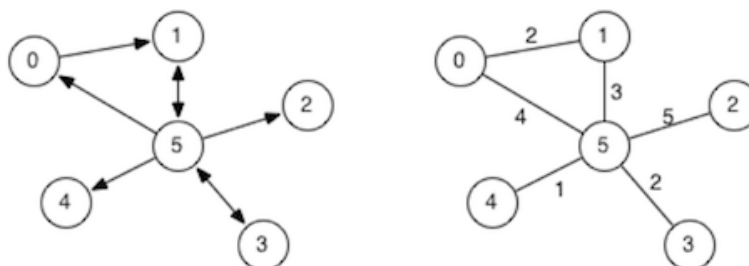
Minimum Spanning Trees, Shortest Paths, Maximum Flows

Please note:

- The contents of this week's lecture and homework will *not* be covered by the mid-term test.
- Because of the mid-term test on Monday, October 28, the deadline for this week's programs will be on the day of the next lecture: *Wednesday, October 30th*.

1. (Graph representations)

- a. Consider the following graphs, where bi-directional edges are depicted as two-way arrows rather than having two separate edges going in opposite directions:



For each of the graphs show the concrete data structures if the graph was implemented via:

- adjacency matrix representation (assume full $V \times V$ matrix)
- adjacency list representation (if non-directional, include both (v,w) and (w,v))

- b. Consider the following map of streets in the Sydney CBD:



Represent this as a directed graph, where intersections are vertices and the connecting streets are edges. Ensure that the directions on the edges correctly reflect any one-way streets (this is a driving map, not a walking map). You only need to make a graph which includes the intersections marked with red letters. Some things that don't show on the map: Castlereagh St is one-way heading south and Curtin Pl is a little laneway that you can't drive down.

For each of the following pairs of intersections, indicate whether there is a path from the first to the second. Show a simple path if there is one. If there is more than one simple path, show two different paths.

- from intersection "D" on Margaret St to intersection "L" on Pitt St
- from intersection "J" to the corner of Margaret St and York St (intersection "A")
- from intersection "P" on Castlereagh St to the corner of Margaret St and Wynyard Ln ("C")
- from the intersection of Castlereagh St and Hunter St ("M") to intersection "H" at Wynyard Park

2. (Warshall's algorithm)

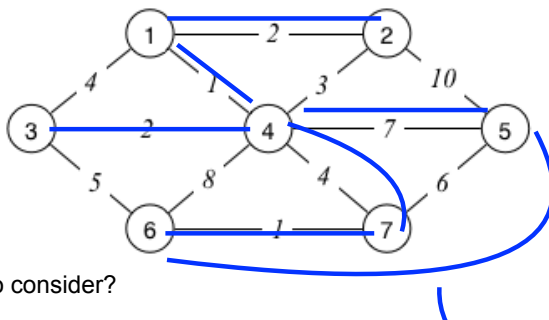
Apply Warshall's algorithm to compute the transitive closure of your graph from exercise 1b. Choose vertices in alphabetical order. Show the reachability matrix:

- after the initialisation
- after the 1st iteration (vertex 'A') of the outermost loop
- after the 6th iteration (vertex 'F') of the outermost loop
- at the end.

Interpret the values in each of these matrices: Which connections between vertices do the different matrices encode?

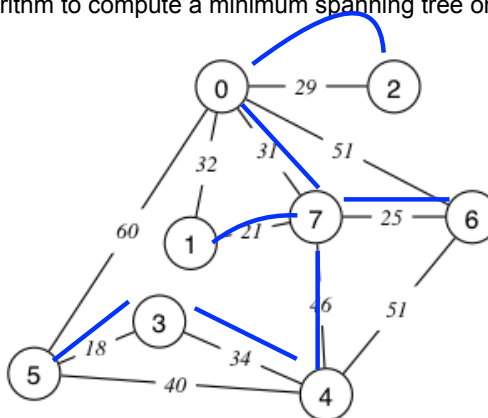
3. (Minimum spanning trees)

- a. Show how Kruskal's algorithm would construct the MST for the following graph:



How many edges do you have to consider?

- b. For a graph $G=(V,E)$, what is the least number of edges that might need to be considered by Kruskal's algorithm, and what is the most number of edges? Add one vertex and edge to the above graph to force Kruskal's algorithm to the worst case.
- c. Trace the execution of Prim's algorithm to compute a minimum spanning tree on the following graph:



Choose a random vertex to start with. Draw the resulting minimum spanning tree.

4. (Priority queue for Dijkstra's algorithm)

Assume that a priority queue for Dijkstra's algorithm is represented by a global variable PQueue of the following structure:

```
#define MAX_NODES 1000
typedef struct {
    int item[MAX_NODES]; // array of vertices currently in queue
    int length;           // #values currently stored in item[] array
} PQueueT;

static PQueueT PQueue; // defines the Priority Queue Object
```

Further assume that vertices are stored in the `item[ ]` array in the priority queue in *no* particular order. Rather, the `item[ ]` array acts like a set, and the priority is determined by reference to a `priority[0..nV-1]` array.

If the priority queue PQueue is initialised by:

```
// set up empty priority queue
void PQueueInit() {
    PQueue.length = 0;
}
```

and new elements are added as follows:

```
// insert vertex v into priority queue
// no effect if v is already in the queue
void joinPQueue(Vertex v) {
    assert(PQueue.length < MAX_NODES);           // ensure queue ADO is not full
    int i = 0;
    while (i < PQueue.length && PQueue.item[i] != v) // check if v already in queue
        i++;
    if (i == PQueue.length) {                      // v not found => add it at the end
        PQueue.item[PQueue.length] = v;
        PQueue.length++;
    }
}
```

then give the implementation of the following functions in C for a priority queue ADO:

```
// remove the highest priority vertex from PQueue
// remember: highest priority = lowest value priority[v]
// returns the removed vertex
Vertex leavePQueue(int priority[]) {
    ...
}

// check if priority queue PQueue is empty
bool PQueueIsEmpty() {
    ...
}
```

Download the ADO header [PQueue.h](#) and the incomplete implementation file [PQueue.c](#) to get you started, along with the Weighted Graph ADT ([WGraph.h](#), [WGraph.c](#)) from the lecture. Do not change the header files or the weighted graph ADT.

Here is a simple [client program](#) to demonstrate the functionality of the priority queue:

```
// PQueue client ... COMP9024 19T3
#include <stdio.h>
#include "PQueue.h"

int main(void) {
    int distance[2] = { 90, 24 };

    PQueueInit();
    joinPQueue(0); // vertex 0 (distance = 90) joins the queue
    joinPQueue(1); // vertex 1 (distance = 24) joins the queue

    while (!PQueueIsEmpty()) {
        printf("Dequeuing %d.\n", leavePQueue(distance));
    }
    return 0;
}
```

The expected output is:

```
prompt$ ./PQueueClient
Dequeuing 1.
Dequeuing 0.
```

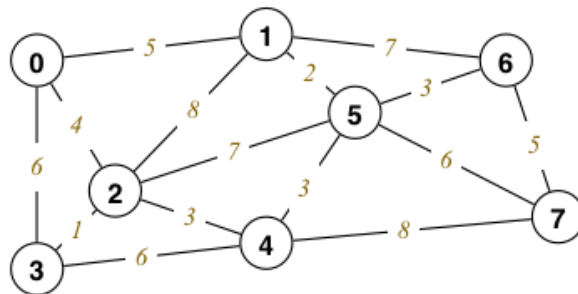
We have created a script that can automatically test your program. To run this test you can execute the `dryrun` program that corresponds to this exercise. It expects to find a file named `PQueue.c` in the current directory that provides an implementation of a priority queue ADO with the four functions shown above (`PQueueInit`, `JoinPQueue`, `LeavePQueue`, `PQueueIsEmpty`).

You can use `dryrun` as follows:

```
prompt$ 9024 dryrun PQueue
```

5. (Dijkstra's algorithm)

- Consider the following graph:



What is the shortest path from vertex 0 to vertex 3? From vertex 0 to vertex 7? How did you determine these?

- b. Trace the execution of Dijkstra's algorithm on the graph from the previous question to compute the minimum distances from source node 0 to all other vertices.

Show the values of `vSet`, `dist[ ]` and `pred[ ]` after each iteration.

- c. Download and complete the incomplete implementation [dijkstra.c](#) of Dijkstra's algorithm that uses your priority queue for exercise 4 and the Weighted Graph ADT ([WGraph.h](#), [WGraph.c](#)) from the lecture. The program:

- prompts the user for the number of nodes in a graph,
- prompts the user for a source node,
- builds a weighted *undirected* graph from user input (adds every input edge in both directions),
- uses the priority ADO to compute and output the distance and a shortest path from the source to every vertex of the graph (**needs to be implemented**).

An example of the program executing is shown below. The input graph consists of a simple triangle along with a fourth, disconnected node.

```
prompt$ ./dijkstra
Enter the number of vertices: 4
Enter the source node: 0
Enter an edge (from): 0
Enter an edge (to): 1
Enter the weight: 42
Enter an edge (from): 0
Enter an edge (to): 2
Enter the weight: 25
Enter an edge (from): 1
Enter an edge (to): 2
Enter the weight: 14
Enter an edge (from): done
Done.
0: distance = 0, shortest path: 0
1: distance = 39, shortest path: 0-2-1
2: distance = 25, shortest path: 0-2
3: no path
```

Note that any non-numeric data can be used to 'finish' the interaction. Test your algorithm with the graph from exercise 5a.

To test your program you can execute the `dryrun` program that corresponds to this exercise. It expects to find two programs in the current directory:

- `dijkstra.c` – your solution to this exercise
- `PQueue.c` – your solution to Exercise 4

You can use `dryrun` as follows:

```
prompt$ 9024 dryrun dijkstra
```

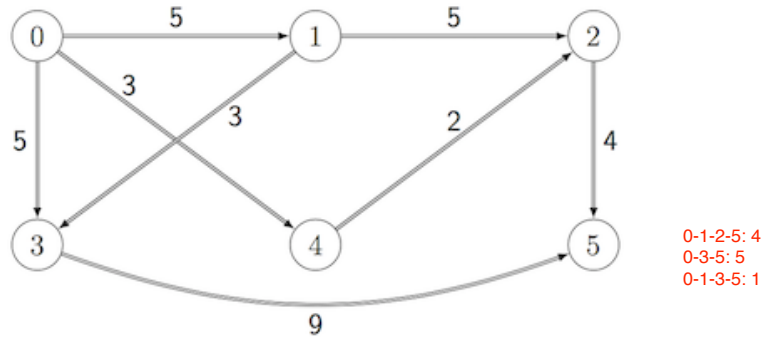
Note: Please ensure that your output follows exactly the format shown above.

6. (All-pair shortest paths)

- a. In the graph from exercise 5a, what is the shortest path from vertex 3 to vertex 6? From vertex 6 to vertex 3? How did you determine these?
- 3-2-5-6
- b. Trace the execution of Floyd's algorithm on the graph from exercise 5a to compute the shortest paths between *all* pairs of vertices.

7. (Maximum flow)

- a. Identify a maximum flow from source=0 to sink=5 in the following network (without applying the algorithm from the lecture):

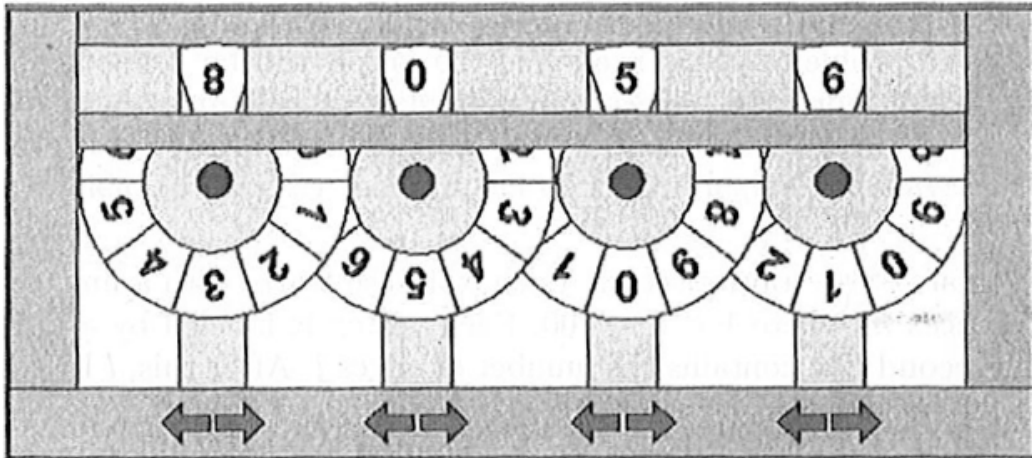


What approach did you use in finding the maxflow?

- b. Show how Edmonds and Karp's algorithm would construct a maximum flow from 0 to 5 for the above network. How many augmenting paths do you have to consider?

8. Challenge Exercise

Consider a machine with four wheels. Each wheel has the digits 0...9 printed clockwise on it. The current *state* of the machine is given by the four topmost digits abcd, e.g. 8056 in the picture below.



Each wheel can be controlled by two buttons: Pressing the button labelled with " $\leftarrow$ " turns the corresponding wheel clockwise one digit ahead, whereas pressing " $\rightarrow$ " turns it anticlockwise one digit back.

Write a C-program to determine the *minimum* number of button presses required to transform

- a given initial state, abcd
- to a given goal state, efgh
- without passing through any of $n \geq 0$ given "forbidden" states, $\text{forbidden}[] = \{w_1x_1y_1z_1, \dots, w_nx_ny_nz_n\}$.

For example, the state 8056 as depicted can be transformed into 0056 in 2 steps if $\text{forbidden}[] = \{ \}$, whereas a minimum of 4 steps is needed for the same task if $\text{forbidden}[] = \{9056\}$. (Why?)

Use your program to compute the least number of button presses required to transform 8056 to 7012 if

- there are no forbidden states
- you are not permitted to pass through any state 7055–8055 (i.e., 7055, 7056, ..., 8055 all are forbidden)
- you are not permitted to pass through any state 0000–0999 or 7055–8055

Assessment

Group submissions will not be allowed. Your program must be entirely your own work. Plagiarism detection software will be used to compare all submissions pairwise.

Submit your solutions to **Exercise 4 and 5** using the following **give** command:

```
prompt$ give cs9024 week6 PQueue.c dijkstra.c
```

Make sure you spell the filenames correctly. You can run **give** multiple times. Only your last submission will be marked.

The deadline for submission is **Wednesday, 30 October 11:00:00am**.

Each program is worth 1 mark. Total marks = 2. Auto-marking will be run by the lecturer several days after the submission deadline using different test cases than `dryrun` does.

Hints:

- Programs will not be manually marked.
- It is important that the output of your program follows exactly the format shown above, otherwise auto-marking will result in 0 marks.
- Ensure that your program compiles on a CSE machine with the standard options `-Wall -Werror -std=c11`. Programs that do not compile will receive 0 marks.
- `dryrun` and auto-marking also check the correct usage of dynamic memory allocation and pointers as well as the absence of memory leaks.
- Do your own testing in addition to running `dryrun`.